

Equações diferenciais

PVI · MDF · BEM 1D · linhas

Lucas S. Campos

Introdução ao MEC / BEM

Equações diferenciais

Gráficos deste capítulo usam **Plots.jl**. Pacotes: DifferentialEquations, Plots, LinearAlgebra.

Objetivos

Ao final, você deve ser capaz de:

1. Formular um PVI escalar e resolvê-lo com `DifferentialEquations` e com Euler / RK4 próprios.
2. Medir ordem de convergência em gráfico log-log.
3. Reduzir uma EDO de 2ª ordem a um sistema de 1ª ordem.
4. Montar matrizes de diferenciação D_x , D_{xx} , impor Dirichlet e resolver um PVC 1D.
5. Integrar no tempo o sistema semi-discreto (método das linhas) para a equação do calor — em MDF e em BEM 1D.
6. Explicar a forma $M\dot{T} + HT = GQ$ e extrair um PVI nas incógnitas de contorno.

Mapa do capítulo

1. PVI e o solucionador Tsit5
2. Euler (ordem 1) e estudo de erro
3. Runge–Kutta 4
4. Sistemas e redução de ordem
5. Matrizes de diferenças finitas e PVC estacionário
6. Método das linhas: difusão (MDF)
7. Equação da onda (método das linhas)
8. Ponte com o BEM e método das linhas no contorno
9. Exercícios
10. (Extra) multi-passo AB4 e Houbolt

PVI escalar

Quantidades que mudam no tempo (ou ao longo de um parâmetro) são modeladas por equações diferenciais. Condições suplementares fecham o problema: no **problema de valor inicial** (PVI) tudo é prescrito em um único valor da variável independente — em geral o tempo.

PVI escalar de primeira ordem:

$$\begin{aligned}u'(t) &= f(t, u(t)), & a \leq t \leq b, \\u(a) &= u_0.\end{aligned}$$

- t : variável independente; u : dependente.
- Se $f(t, u) = g(t) + h(t)u$, a EDO é **linear**; caso contrário, **não linear**.
- Solução: função $u(t)$ que satisfaz a EDO e a condição inicial.
- Notação no tempo: às vezes $\dot{u}$ em vez de u' .

Ideia-chave. Métodos de PVI **avançam** a partir de $u(a)$: a cada passo usam f (a inclinação) para construir u em tempos futuros. A qualidade depende da **ordem**, da **estabilidade** e do tamanho de passo h .

Exemplo: $u' = \sin((u + t)^2)$, $t \in [0, 4]$, $u(0) = 1$.

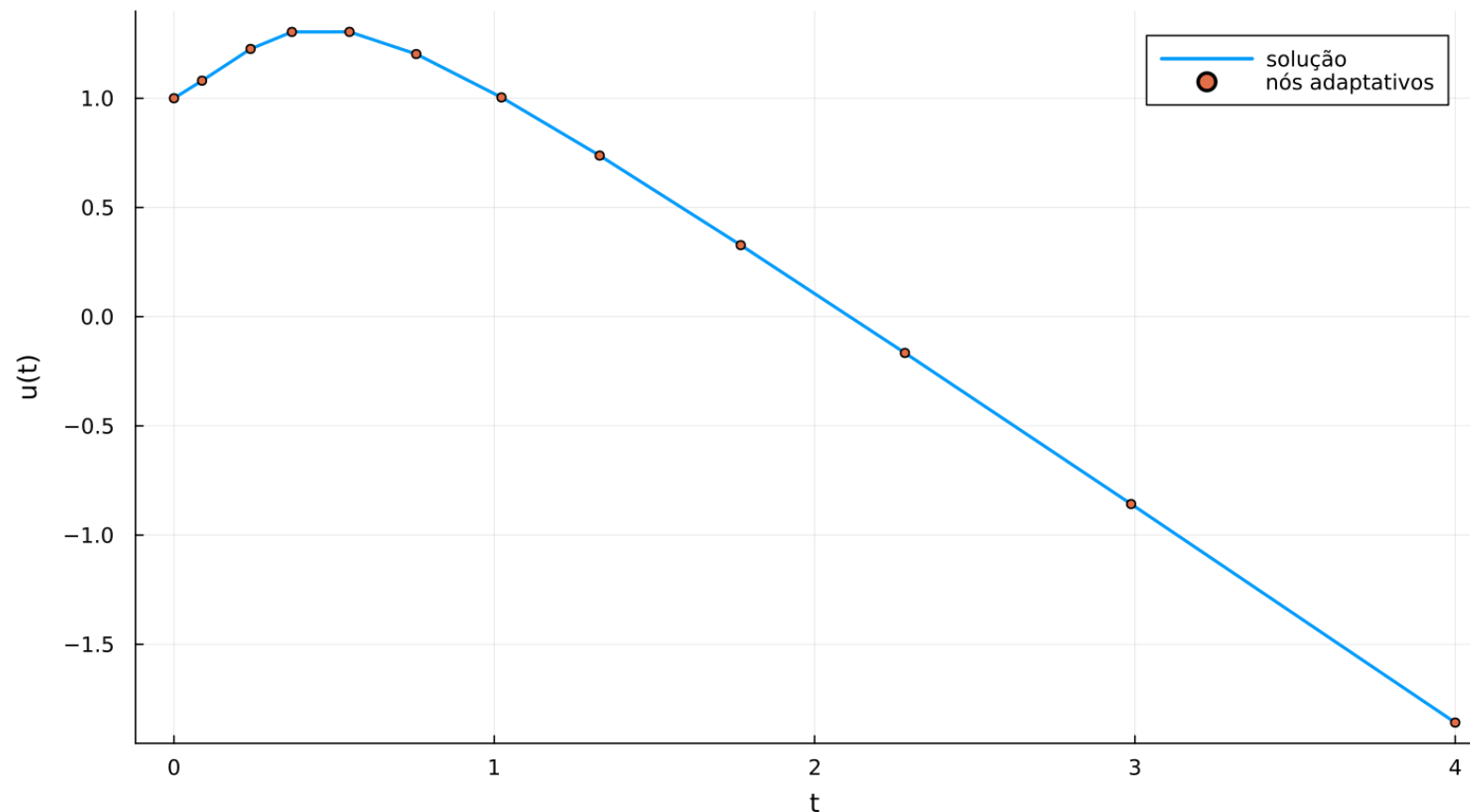
A API pede $f(u, p, t)$ — o argumento p carrega parâmetros constantes (mesmo que não usemos).

```
using DifferentialEquations, Plots, LinearAlgebra

f = (u, p, t) -> sin((t + u)^2)
u0 = 1.0
tspan = (0.0, 4.0)

ivp = ODEProblem(f, u0, tspan)
sol = solve(ivp, Tsit5())

plot(sol.t, sol.u; xlabel="t", ylabel="u(t)", label="solução", lw=2)
scatter!(sol.t, sol.u; label="nós adaptativos", ms=3)
```



O objeto `sol` é avaliável em qualquer t (`sol(1.0)`): por baixo há uma malha adaptativa e interpolação. O restante do capítulo explica **como** se constroem valores discretos (t_i, u_i) — com passo fixo, para controlar ordem.

Método de Euler

Discretize o tempo em passos iguais:

$$t_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

Seja $\hat{u}(t)$ a solução **exata** e $u_i \approx \hat{u}(t_i)$ a aproximação numérica.

Ideia: no intervalo $[t_i, t_{i+1}]$, a inclinação do interpolante linear por partes é $\frac{u_{i+1}-u_i}{h}$. Iguale a $f(t_i, u_i)$:

$$u_{i+1} = u_i + hf(t_i, u_i).$$

Isso é o **método de Euler explícito** (ordem 1).

```
"""
    euler(ivp, n) -> t, U

Euler explícito com n passos. `U[i]` é o estado em t[i]
(escalar ou vetor, conforme `ivp.u0`).
"""
function euler(ivp, n)
    a, b = ivp.tspan
    h = (b - a) / n
    t = [a + i*h for i in 0:n]
    u0 = ivp.u0 isa Number ? float(ivp.u0) : float.(ivp.u0)
```

```
U = Vector{typeof(u0)}(undef, n + 1)
U[1] = u0
for i in 1:n
    U[i+1] = U[i] + h * ivp.f(U[i], ivp.p, t[i])
end
return t, U
end
```

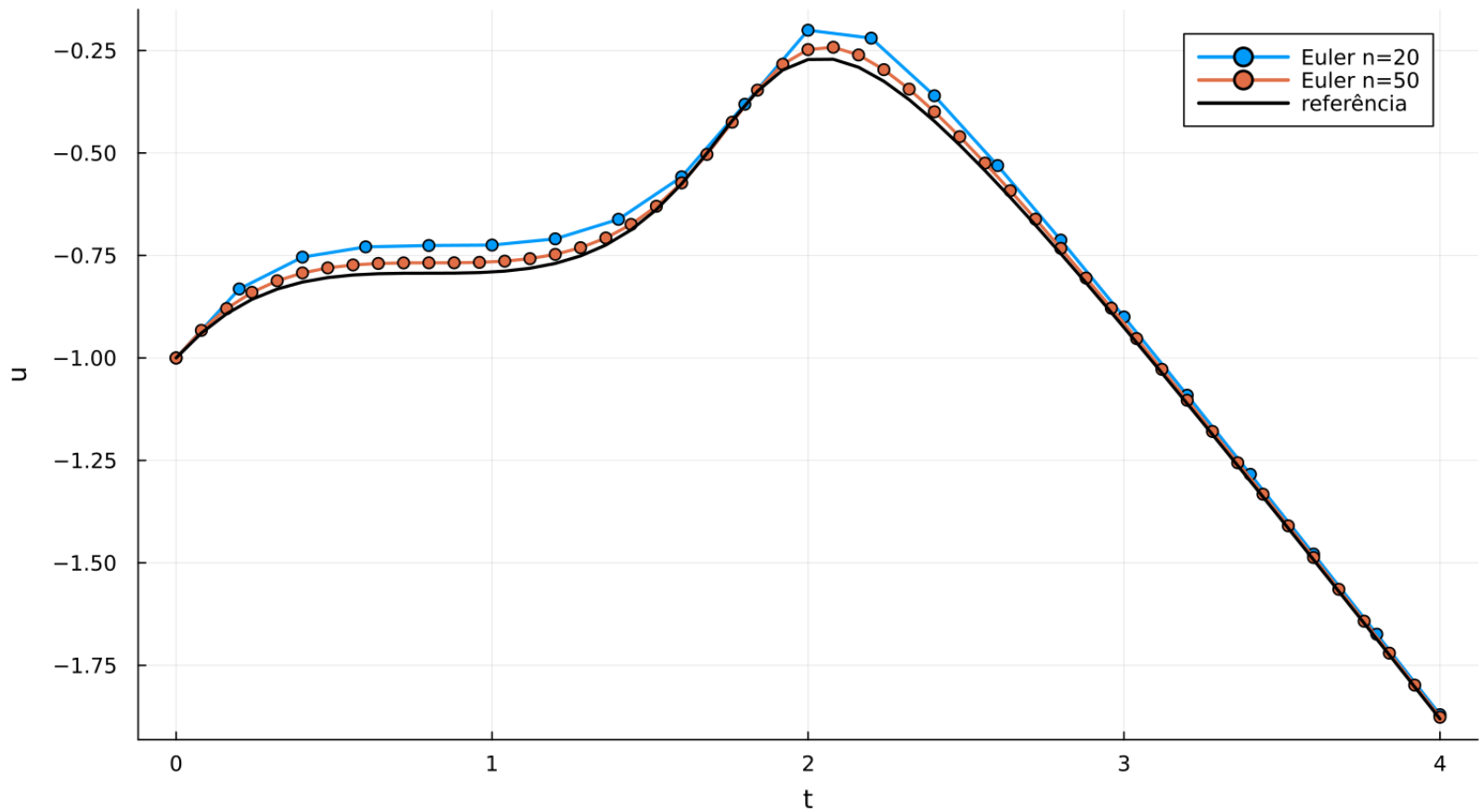
Exemplo e convergência

Mesma EDO, agora $u(0) = -1$:

```
f = (u, p, t) -> sin((t + u)^2)
ivp = ODEProblem(f, -1.0, (0.0, 4.0))

t20, u20 = euler(ivp, 20)
t50, u50 = euler(ivp, 50)
u_ref = solve(ivp, Tsit5(); reltol=1e-14, abstol=1e-14)

plot(t20, u20; marker=:circle, label="Euler n=20", xlabel="t", ylabel="u", lw=2)
plot!(t50, u50; marker=:circle, label="Euler n=50", lw=2)
plot!(t50, u_ref.(t50); color=:black, lw=2, label="referência")
```

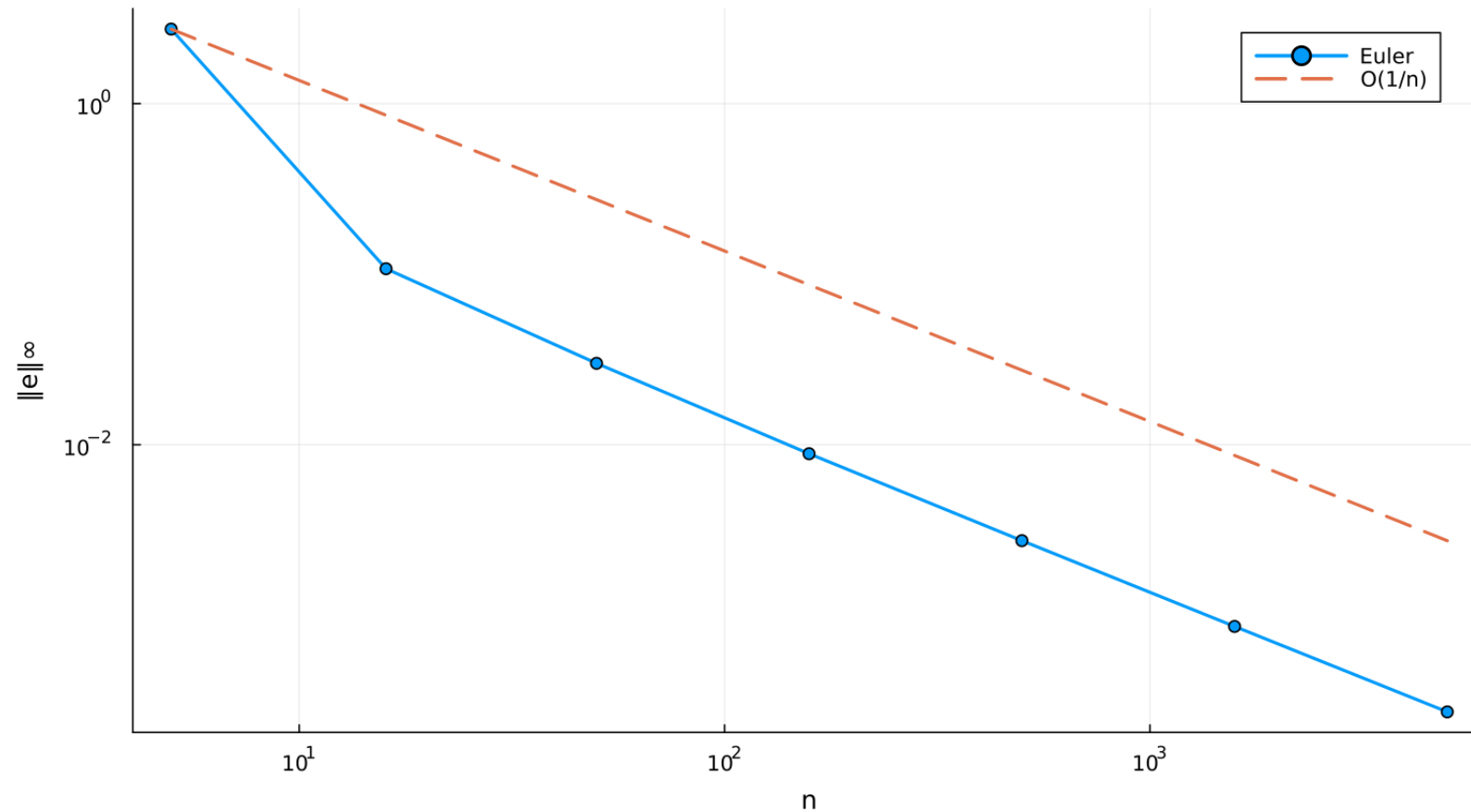


Estudo de erro

Estudo de erro em norma do máximo nos nós:

```
ns = [round(Int, 5 * 10^k) for k in 0:0.5:3]
err_E = Float64[]
for n in ns
    t, U = euler(ivp, n)
    push!(err_E, maximum(abs, u_ref.(t) .- U))
end

plot(ns, err_E; xscale=:log10, yscale=:log10, marker=:circle,
      label="Euler", xlabel="n", ylabel="||e||∞", lw=2)
plot!(ns, err_E[1] .* (ns[1] ./ ns); ls=:dash, label="O(1/n)")
```



Ideia-chave — ordem. Se o erro global se comporta como $O(h^p) = O(n^{-p})$, no plano $\log n \times \log \text{erro}$ a reta tem inclinação $-p$. Euler tem $p = 1$: multiplicar n por 10 reduz o erro cerca de 10×.

Runge-Kutta de 4ª ordem

Euler usa **uma** avaliação de f por passo. Métodos de Runge–Kutta (RK) combinam várias avaliações (**estágios**) no intervalo $[t_i, t_{i+1}]$ para subir a ordem.

O RK clássico de ordem 4:

$$\begin{aligned}k_1 &= hf(t_i, u_i), \\k_2 &= hf\left(t_i + \frac{h}{2}, u_i + \frac{k_1}{2}\right), \\k_3 &= hf\left(t_i + \frac{h}{2}, u_i + \frac{k_2}{2}\right), \\k_4 &= hf(t_i + h, u_i + k_3), \\u_{i+1} &= u_i + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}.\end{aligned}$$

```
"""RK4 clássico, n passos. Aceita estado escalar ou vetorial."""
function rk4(ivp, n)
    a, b = ivp.tspan
    h = (b - a) / n
    t = [a + i*h for i in 0:n]
    u0 = ivp.u0 isa Number ? float(ivp.u0) : float.(ivp.u0)
    U = Vector{typeof(u0)}(undef, n + 1)
    U[1] = u0
    for i in 1:n
```

Runge-Kutta de 4ª ordem

```

    ui, ti = U[i], t[i]
    k1 = h * ivp.f(ui,          ivp.p, ti)
    k2 = h * ivp.f(ui + k1/2,   ivp.p, ti + h/2)
    k3 = h * ivp.f(ui + k2/2,   ivp.p, ti + h/2)
    k4 = h * ivp.f(ui + k3,      ivp.p, ti + h)
    U[i+1] = ui + (k1 + 2k2 + 2k3 + k4)/6
end
return t, U
end

```

Compare ordens no mesmo PVI:

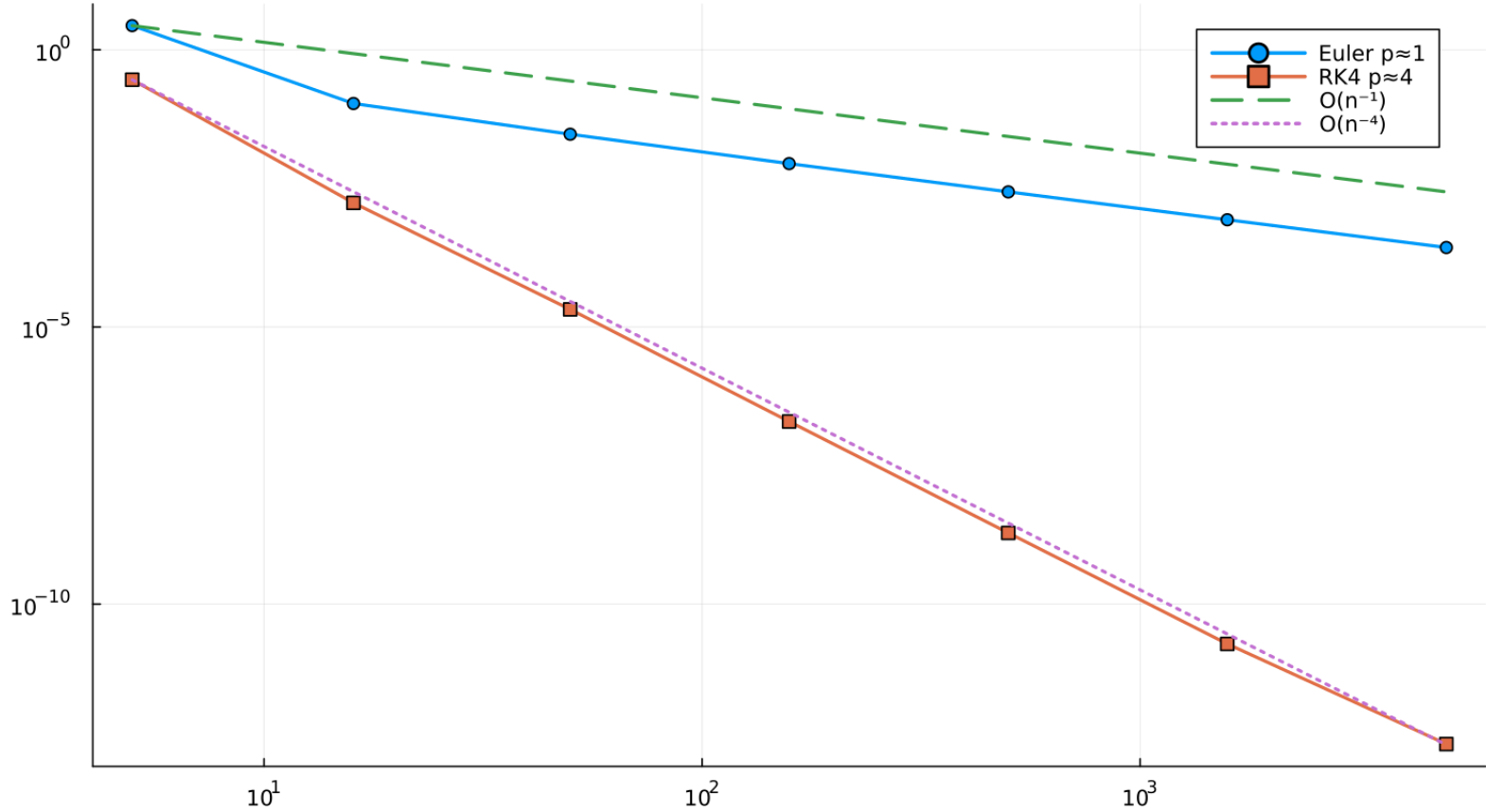
```

err_R = Float64[]
for n in ns
    t, U = rk4(ivp, n)
    push!(err_R, maximum(abs, u_ref.(t) .- U))
end

plot(ns, err_E; xscale=:log10, yscale=:log10, marker=:circle, label="Euler p≈1", lw=2)
plot!(ns, err_R; marker=:square, label="RK4 p≈4", lw=2)
plot!(ns, err_E[1] .* (ns[1] ./ ns); ls=:dash, label="O(n⁻¹)")
plot!(ns, err_R[1] .* (ns[1] ./ ns).^4; ls=:dot, label="O(n⁻⁴)")

```

Euler vs RK4

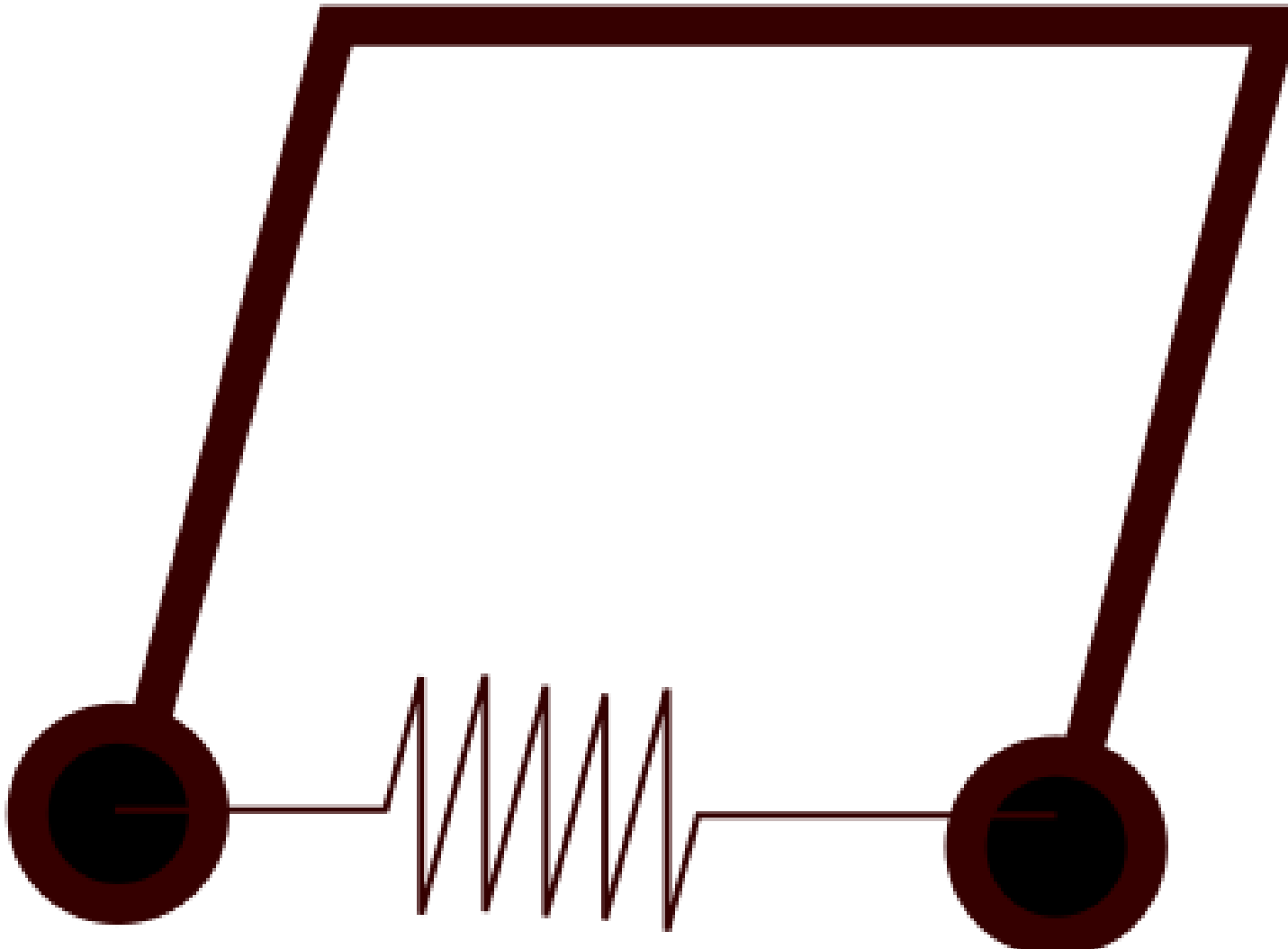


Euler melhorado (RK de ordem 2, um estágio intermediário em $t_i + h/2$) fica como leitura opcional; a trilha do curso usa Euler (referência de ordem 1) e RK4.

Poucas aplicações são escalares. O mesmo Euler/RK4 vale para $\mathbf{u} \in \mathbb{R}^d$ se f devolver um vetor:

$$\mathbf{u}_{i+1} = \mathbf{u}_i + hf(t_i, \mathbf{u}_i) \quad (\text{Euler})$$

EDOs de ordem m viram sistemas de 1ª ordem introduzindo derivadas inferiores como novas incógnitas.



Pêndulos acoplados (2ª ordem -> 1ª)

$$\theta_{1''} + \gamma\theta_{1'} + (g/L) \sin \theta_1 + k(\theta_1 - \theta_2) = 0,$$

$$\theta_{2''} + \gamma\theta_{2'} + (g/L) \sin \theta_2 + k(\theta_2 - \theta_1) = 0.$$

Com $u = (\theta_1, \theta_2, \theta_{1'}, \theta_{2'})$:

$$u_{1'} = u_3, \quad u_{2'} = u_4,$$

$$u_{3'} = -\gamma u_3 - (g/L) \sin u_1 + k(u_2 - u_1),$$

$$u_{4'} = -\gamma u_4 - (g/L) \sin u_2 + k(u_1 - u_2).$$

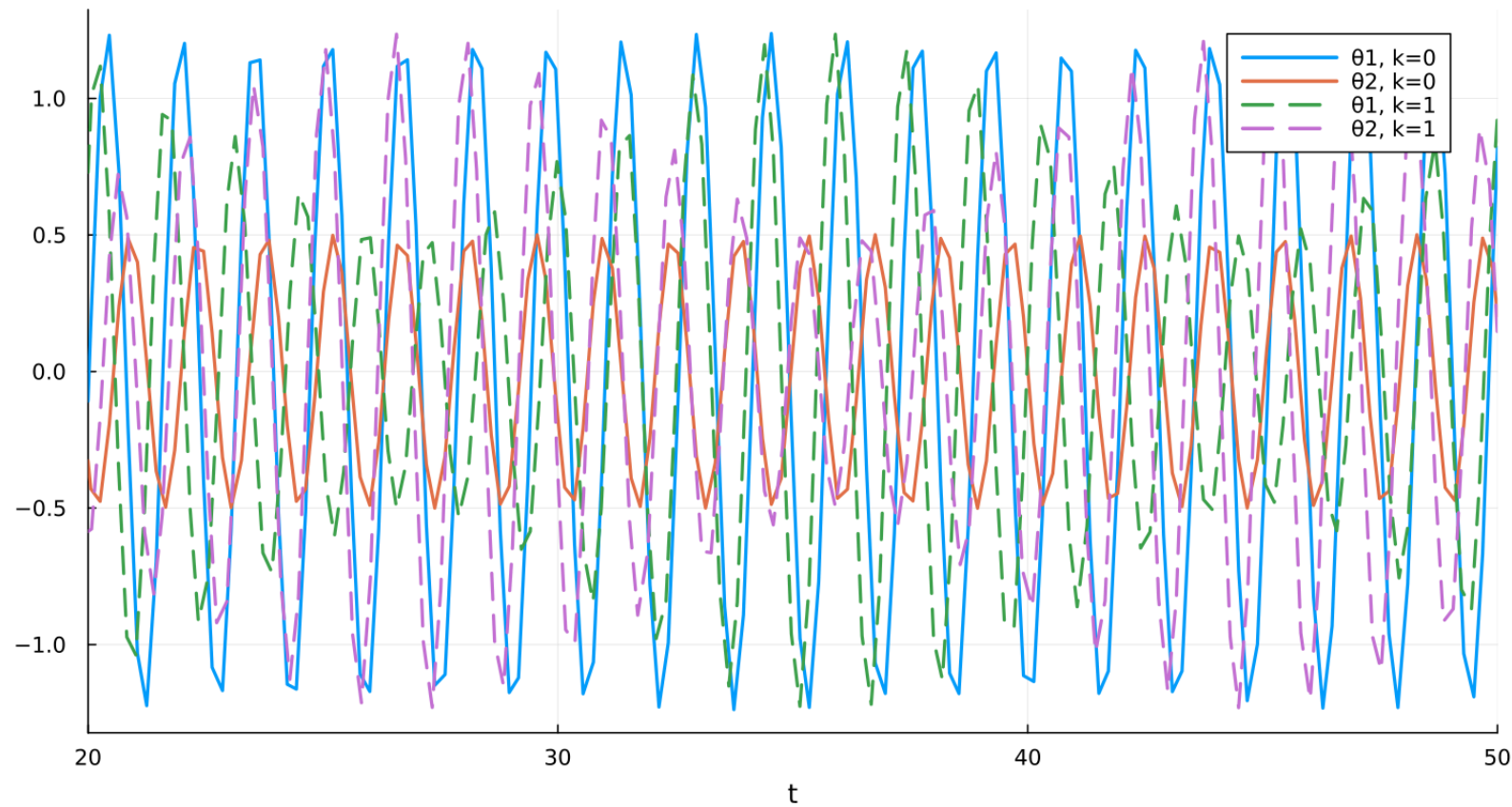
```
function couple(u, p, t)
    γ, L, k = p
    g = 9.8
    du = similar(u)
    du[1] = u[3]
    du[2] = u[4]
    du[3] = -γ*u[3] - (g/L)*sin(u[1]) + k*(u[2] - u[1])
    du[4] = -γ*u[4] - (g/L)*sin(u[2]) + k*(u[1] - u[2])
    return du
end

u0 = [1.25, -0.5, 0.0, 0.0]
tspan = (0.0, 50.0)
γ, L = 0.0, 0.5
```

```
sol0 = solve(ODEProblem(couple, u0, tspan, [γ, L, 0.0]), Tsit5())
sol1 = solve(ODEProblem(couple, u0, tspan, [γ, L, 1.0]), Tsit5())

plot(sol0.t, [u[1] for u in sol0.u]; label="θ1, k=0", xlims=(20, 50), lw=2)
plot!(sol0.t, [u[2] for u in sol0.u]; label="θ2, k=0", lw=2)
plot!(sol1.t, [u[1] for u in sol1.u]; label="θ1, k=1", ls=:dash, lw=2)
plot!(sol1.t, [u[2] for u in sol1.u]; label="θ2, k=1", ls=:dash, lw=2,
      xlabel="t", title="Pêndulos acoplados")
```

Pêndulos acoplados



Receita. Para cada variável de ordem máxima m , crie m componentes $(y, y', \dots, y^{(m-1)})$. Relações cinemáticas $y' = v$, etc., + a EDO original fecham o sistema. Dimensão = soma das ordens.

Matrizes de diferenças finitas

Até aqui a variável independente era o **tempo**. No espaço, para um PVC ou para semi-discretizar uma EDP, aproximamos d/dx e d^2/dx^2 por **matrizes de diferenciação**.

Nós uniformes em $[a, b]$:

$$x_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

Queremos $\mathbf{g} \approx f'(\mathbf{x})$ com $\mathbf{g} = D_x \mathbf{f}$. Usamos diferenças centradas de ordem 2 no interior e unilaterais de ordem 2 nos extremos:

$$D_x = \frac{1}{h} \begin{pmatrix} -3/2 & 2 & -1/2 & & \\ -1/2 & 0 & 1/2 & & \\ & \ddots & \ddots & \ddots & \\ & & -1/2 & 0 & 1/2 \\ & & 1/2 & -2 & 3/2 \end{pmatrix}.$$

Segunda derivada (ordem 2):

$$D_{xx} = \frac{1}{h^2} \begin{pmatrix} 2 & -5 & 4 & -1 & \\ 1 & -2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2 & 1 \\ -1 & 4 & -5 & 2 & \end{pmatrix}.$$

```
"""
    diffmat(n, xspan) -> x, Dx, Dxx

n = número de *intervalos* (n+1 nós) em xspan = (a,b).
Diferenças de ordem 2 (centradas no interior).
"""
function diffmat(n, xspan)
    a, b = xspan
    h = (b - a) / n
    x = [a + i*h for i in 0:n]
    # Dx
    dp = fill(0.5/h, n)
    dm = fill(-0.5/h, n)
    Dx = diagm(-1 => dm, 1 => dp)
    Dx[1, 1:3] = [-1.5, 2, -0.5] / h
    Dx[n+1, n-1:n+1] = [0.5, -2, 1.5] / h
    # Dxx
    d0 = fill(-2/h^2, n+1)
    dp2 = ones(n) / h^2
    Dxx = diagm(-1 => dp2, 0 => d0, 1 => dp2)
    Dxx[1, 1:4] = [2, -5, 4, -1] / h^2
    Dxx[n+1, n-2:n+1] = [-1, 4, -5, 2] / h^2
```

```
    return x, Dx, Dxx  
end
```

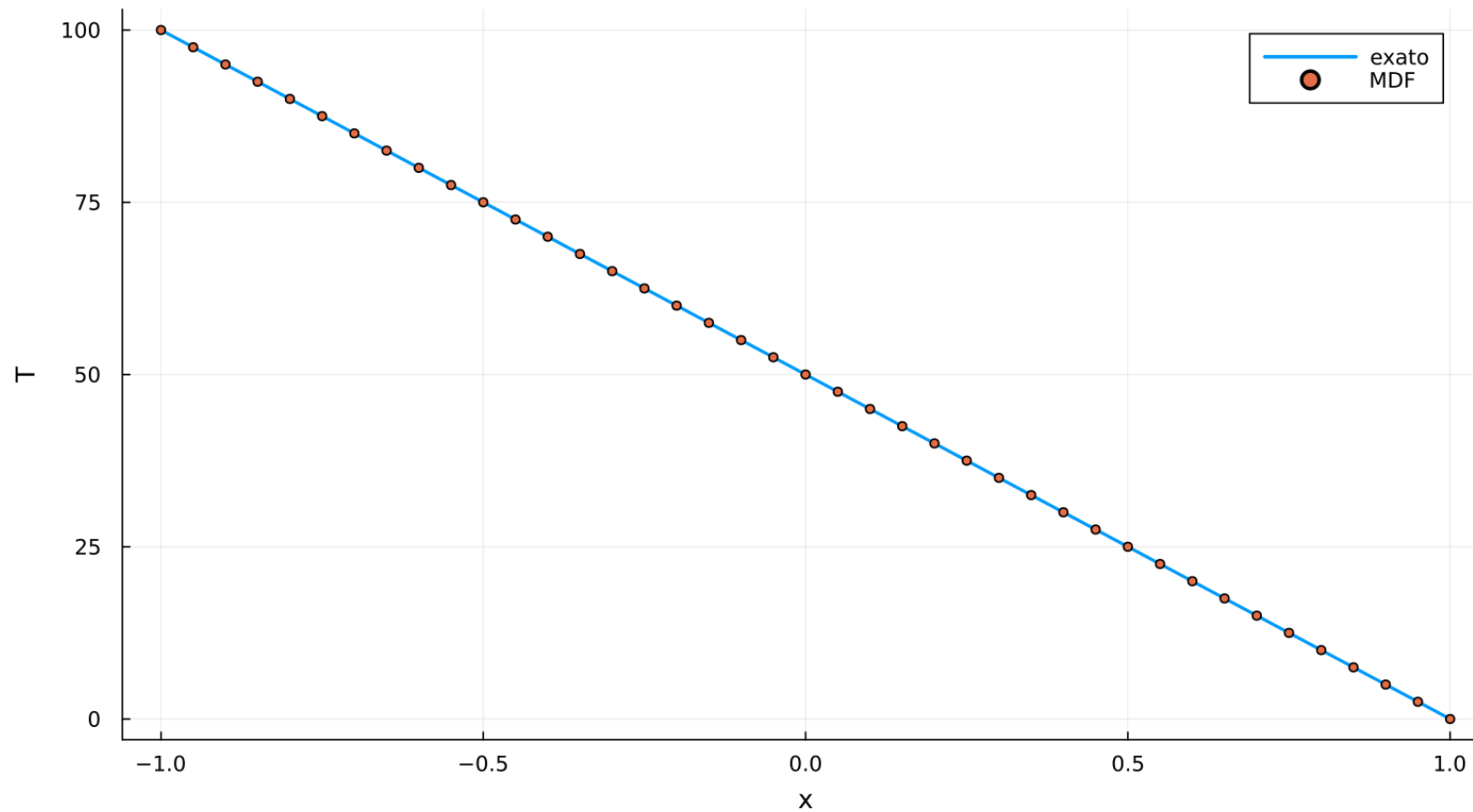
PVC estacionário (Laplace 1D)

$T''(x) = 0$ em $(-1, 1)$, $T(-1) = 100$, $T(1) = 0$. Solução exata: $T(x) = 50(1 - x)$.

Impor Dirichlet **substituindo linhas** de D_{xx} :

```
n = 40
x, Dx, Dxx = diffmat(n, (-1.0, 1.0))
A = copy(Dxx)
A[1, :] .= 0; A[1, 1] = 1
A[end, :] .= 0; A[end, end] = 1
rhs = zeros(n + 1)
rhs[1] = 100
rhs[end] = 0
T = A \ rhs
T_ex = @. 50 * (1 - x)

plot(x, T_ex; label="exato", lw=2, xlabel="x", ylabel="T")
scatter!(x, T; label="MDF", ms=3)
@show maximum(abs, T - T_ex)
```



Ideia-chave. $D_{xx}\mathbf{T} = \mathbf{b}$ é um sistema algébrico esparsos nos nós do **domínio**. No BEM 1D da apresentação, o sistema vive só nas **pontas** e usa $HT = GQ$. Mesma física, densidades de informação diferentes.

Método das linhas: equação do calor

EDP parabólica 1D:

$$u_t = \kappa u_{xx},$$

com $\kappa > 0$. Fixe a malha em x , substitua $u_{xx} \approx D_{xx} \mathbf{u}(t)$ e obtenha um **PVI grande** no tempo — o **método das linhas**.

Condições: $u(-1, t) = 0$, $u(1, t) = 2$, e

$$u(x, 0) = 1 + \sin(\pi x/2) + 3(1 - x^2)e^{-4x^2}.$$

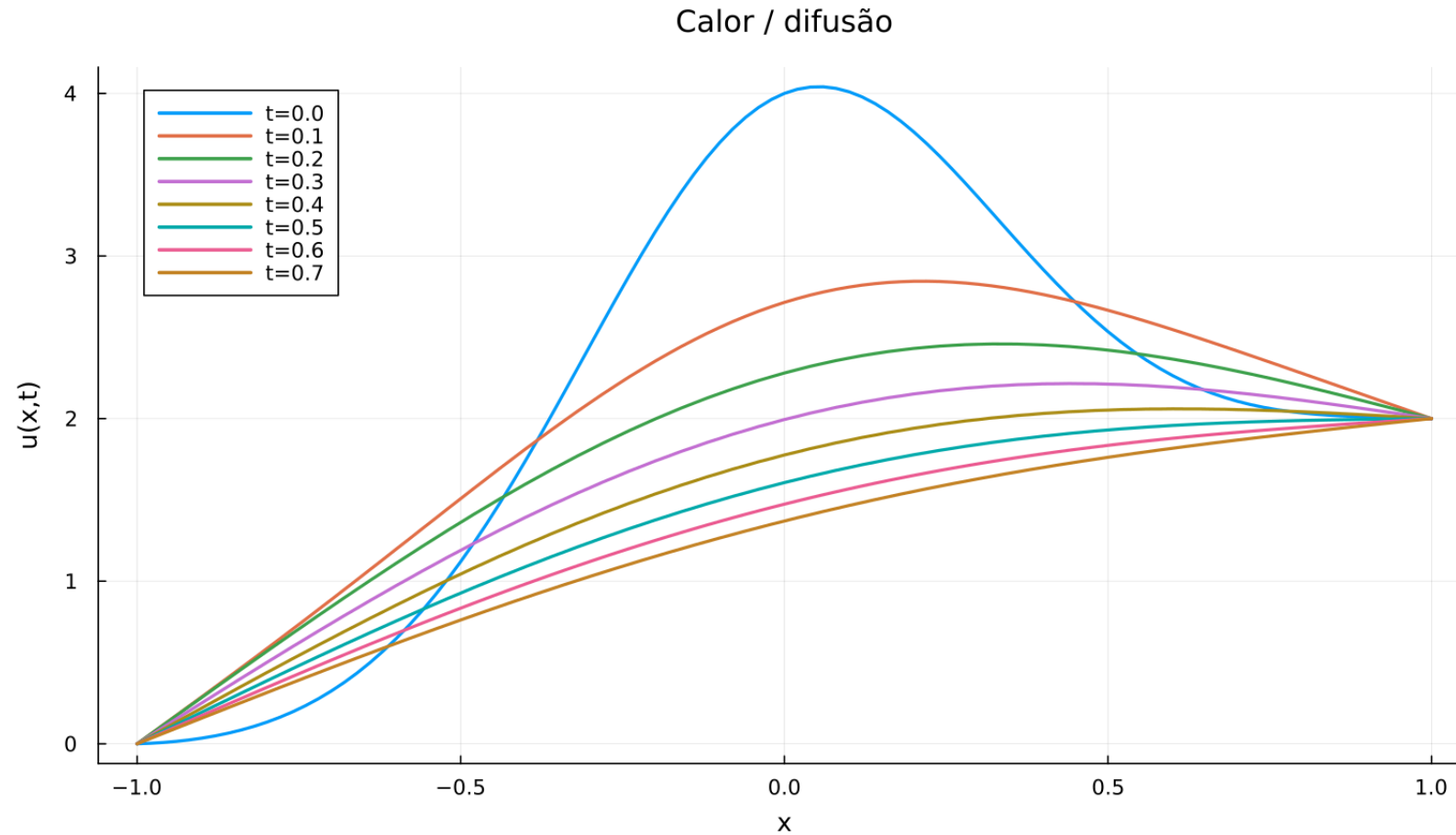
```
function heat_rhs!(du, u, p, t)
    Dxx, κ, ua, ub = p
    u[1] = ua
    u[end] = ub
    mul!(du, Dxx, u)
    du .*= κ
    du[1] = 0
    du[end] = 0
    return nothing
end

n = 100
x, Dx, Dxx = diffmat(n, (-1.0, 1.0))
κ = 1.0
ua, ub = 0.0, 2.0
```

```
u0 = @. 1 + sin(pi*x/2) + 3*(1 - x^2)*exp(-4*x^2)
u0[1] = ua; u0[end] = ub

prob = ODEProblem(heat_rhs!, u0, (0.0, 0.75), (Dxx, κ, ua, ub))
sol = solve(prob, Tsit5())

plt = plot(xlabel="x", ylabel="u(x,t)", title="Calor / difusão", legend=:topleft)
for tt in 0:0.1:0.7
    plot!(plt, x, sol(tt); label="t=$tt", lw=2)
end
plt
```



Animação opcional (GIF local) ou vídeo do repositório:

```
anim = @animate for tt in range(0, 0.75; length=60)
    plot(x, sol(tt); xlabel="x", ylabel="u", ylims=(0, 4.2),
         legend=false, lw=2, title="t=$(round(tt; digits=3))")
end
gif(anim, "calor.gif"; fps=15)
```

Vídeo pronto: boundaries-heat.mp4

Os integradores do início do capítulo (Euler, RK4, Tsit5) aplicam-se a esse sistema; a estabilidade explícita exige $\Delta t = O(h^2)$ para o calor — por isso solucionadores **adaptativos** ou implícitos ajudam.

EDP hiperbólica 1D (corda / acústica unidimensional):

$$u_{tt} = c^2 u_{xx},$$

com velocidade de onda $c > 0$. Diferente do calor, a informação propaga com **velocidade finita** c e a energia (em domínio isolado) se conserva no contínuo.

Redução a 1ª ordem no tempo

Como no pêndulo, introduza a velocidade $v = u_t$:

$$u_t = v, \quad v_t = c^2 u_{xx}.$$

No espaço, $u_{xx} \approx D_{xx} \mathbf{u}$. O estado do PVI fica o **par** de vetores

$$\mathbf{y} = \begin{pmatrix} \mathbf{u} \\ \mathbf{v} \end{pmatrix} \in \mathbb{R}^{2(n+1)}$$

(ou só nós interiores, se as BC fixarem u nos extremos). Então

$$\dot{\mathbf{y}} = \begin{pmatrix} \mathbf{v} \\ c^2 D_{xx} \mathbf{u} \end{pmatrix}$$

com BC aplicadas em $\mathbf{u}$ (e $\dot{u} = v = 0$ nos extremos se Dirichlet homogêneo constante).

Checklist para implementar.

1. $x, Dx, Dxx = \text{diffmat}(n, (a,b))$.
2. Estado $y = \text{vcat}(u, v)$ com $\text{length}(u) = \text{length}(v) = n+1$.
3. No RHS: separar $u = y[1:n+1]$, $v = y[n+2:\text{end}]$; impor BC em u (e zerar v nos nós de Dirichlet fixo); $du = v$, $dv = c^2 * (Dxx * u)$.
4. Integrar com Tsit5 (ou RK4 próprio).

5. CFL prático: com passo **fixo** explícito, $\Delta t \leq h/c$; com Tsit5 adaptativo o passo se ajusta, mas malha grossa demais ainda dispersa a onda.

Problema-modelo com solução exata

$$u_{tt} = c^2 u_{xx}, \quad 0 < x < 1, \quad t > 0,$$

$$u(0, t) = u(1, t) = 0,$$

$$u(x, 0) = \sin(\pi x), \quad u_t(x, 0) = 0.$$

Modo normal: $u_{\text{exata}}(x, t) = \sin(\pi x) \cos(c\pi t)$ (verifique BC, CI e a EDP).

Esqueleto MDF:

```
function wave_rhs!(dy, y, p, t)
    Dxx, c2, nnode = p
    u = @view y[1:nnode]
    v = @view y[nnode+1:end]
    du = @view dy[1:nnode]
    dv = @view dy[nnode+1:end]
    # Dirichlet homogêneo
    u[1] = 0.0
    u[end] = 0.0
    v[1] = 0.0
    v[end] = 0.0
    du .= v
    mul!(dv, Dxx, u)
    dv .*= c2
    du[1] = 0.0
```

```

    du[end] = 0.0
    dv[1] = 0.0
    dv[end] = 0.0
    return nothing
end

function solve_wave_mdf(; n=100, c=1.0, tspan=(0.0, 2.0))
    x, Dx, Dxx = diffmat(n, (0.0, 1.0))
    nnode = length(x)
    u0 = sin(pi .* x)
    v0 = zeros(nnode)
    y0 = vcat(u0, v0)
    prob = ODEProblem(wave_rhs!, y0, tspan, (Dxx, c^2, nnode))
    sol = solve(prob, Tsit5(); reltol=1e-8, abstol=1e-8)
    return sol, x
end

```

BEM (só a **forma**, para o exercício): no contorno, algo como

$$M\mathbf{u}'' + H\mathbf{u} = G\mathbf{q}$$

com q ligado a $\partial_n u$; o método das linhas no contorno seria um sistema de 2ª ordem no tempo (ou 1ª ordem em $(u, \dot{u})$ nas faces). No E5 pede-se o MDF completo e apenas o esboço matricial BEM.

A equação do calor 1D $T_t = \kappa T_{xx}$ pode ser atacada de dois jeitos depois de discretizar o **espaço**:

Aspecto	MDF (domínio)	BEM (contorno)
Incógnitas primárias	T em todos os nós de $[x_0, x_f]$	T e/ou $Q = \partial T / \partial x$ só em x_0, x_f
Operador espacial	D_{xx} esparsa	matrizes cheias H, G da identidade integral
Tempo	$\dot{\mathbf{T}} = \kappa D_{xx} \mathbf{T}$ (+ BC)	$M \dot{\mathbf{T}} + H \mathbf{T} = G \mathbf{Q}$
Interior	já está no vetor de estado	pós-processamento pela fórmula de representação

Ideia-chave. O **método das linhas** é o mesmo em ambos: depois do espaço, sobra um PVI (ou DAE) no tempo. O que muda é **quem** são as incógnitas e **quais** matrizes multiplicam T , Q e $\dot{T}$.

Na aula 1 (estacionário), $HT = GQ$ nas pontas ($n_c = 2$). Com $T_t = \kappa T''$ vem o domínio

$$\int_{x_0}^{x_f} T^*(x, x_d), \left(\dot{T}(x) / \kappa \right) dx.$$

Cada Gauss é fonte interior: bloco

Gauss–Legendre em $[x_0, x_f]$: nós x_j^g , pesos w_j ($j = 1..n_i$). Cada x_j^g tem **dois** papéis:

1. ponto de **campo** na quadratura de $\dot{T}$;
2. ponto **fonte** (colocação interior, $c = 1$).

Colocações:

$$x_d = (x_0, x_f, x_1^g, \dots, x_{n_i}^g), \quad N = n_c + n_i.$$

Aproximação nodal por Gauss:

$$\int T^*(\dot{T}/\kappa) dx \approx \sum_{j=1}^{n_i} (w_j/\kappa) T^*(x_j^g, x_d^{\{(p)\}}) \dot{T}(x_j^g).$$

Isso define o bloco retangular $M^g \in \mathbb{R}^{N \times n_i}$. O sistema **quadrado** $(n_c + n_i) \times (n_c + n_i)$ do método das linhas fecha com as incógnitas de contorno livres (Q e/ou $\dot{T}$ nos extremos não prescritos) empilhadas às $\dot{T}^g$:

$$z = (Q_0, \dot{T}_f, \dot{T}(x_1^g), \dots, \dot{T}(x_{n_i}^g)) \in \mathbb{R}^N$$

no problema misto T_0 fixo ($\dot{T}_0 = 0$) e $Q_f = 0$. As N colocações fornecem $Az = r$.

Ideia-chave. Montar M **não** é “somar T^* nos Gauss e jogar numa diagonal”. Cada Gauss gera uma **linha** (fonte interior) e uma **coluna** (amostra de $\dot{T}$).

```
using LinearAlgebra, Plots, FastGaussQuadrature, DifferentialEquations

Tstar(x, xd) = -0.5 * abs(x - xd)
Qstar(x, xd) = x == xd ? 0.0 : -0.5 * sign(x - xd)

function bem1d_HG(x0, xf)
    L = xf - x0
    H = [0.5 -0.5; -0.5 0.5]
    G = L * [0.0 -0.5; 0.5 0.0]
    return H, G, L
end

"""
    bem1d_M(x0, xf; κ=1.0, nq=6)

BEM 1D com `nq` Gauss como fontes/interiores.
N = nc + ni = 2 + nq colocações `xd = [x0, xf, xg...]'`.

Retorna:
- `Mg` : N×ni,  Mg[p,j] = w[j]/κ * T*(xg[j], xd[p])
- `Hb, Gb` : N×2 (contorno clássico nas linhas 1:2; representação nas interiores)
"""
```

```
function bem1d_M(x0, xf; κ=1.0, nq=6)
    nc = 2
    ξ, ŵ = gausslegendre(nq)
    jac = (xf - x0) / 2
    xg = collect(@. (x0 + xf)/2 + jac * ξ)
    w = collect(ŵ .* jac)
    ni = nq
    xd = vcat([x0, xf], xg)
    N = nc + ni

    Mg = zeros(N, ni)
    for p in 1:N, j in 1:ni
        Mg[p, j] = w[j] / κ * Tstar(xg[j], xd[p])
    end

    H, G, _ = bem1d_HG(x0, xf)
    Hb = zeros(N, 2)
    Gb = zeros(N, 2)
    Hb[1:2, :] .= H
    Gb[1:2, :] .= G
    # interior p: T(xd) + Hb·Tb - Gb·Qb + Mg·Tg = 0
    # partindo de
```

```
# -T = Tf Q*(xf) - T*(xf) Qf - T0 Q*(x0) + T*(x0) Q0 + ∑ Mg Ṫg
for p in 3:N
    xdp = xd[p]
    Hb[p, 1] = -Qstar(x0, xdp)
    Hb[p, 2] = Qstar(xf, xdp)
    Gb[p, 1] = -Tstar(x0, xdp)
    Gb[p, 2] = Tstar(xf, xdp)
end
return (; x0, xf, xg, w, xd, Mg, Hb, Gb, nc, ni, N, κ)
end
```

Equações em cada colocação $p = 1..N$ (vetor $T^b = (T_0, T_f)$, $Q^b = (Q_0, Q_f)$, $T^g, \dot{T}^g$):

- contorno ($p = 1, 2$): $(HT^b - GQ^b)_p + (M^g \dot{T}^g) = 0$;
- interior ($p = 2 + i$): $T_i^g + (H_{p,:}^b T^b - G_{p,:}^b Q^b) + (M^g \dot{T}^g)_p = 0$.

Incógnitas (BC mista: T_0 fixo, $Q_f = 0$). O sistema **reduzido** de ordem $N - 1$ resolve $z = (Q_0, \dot{T}^g) \in \mathbb{R}^{n_i+1}$ com as colocações em x_0 e em todos os Gauss (linhas 1 e 3.. N). Como o Gauss aberto não coloca massa em x_f , define-se $\dot{T}_f$ pelo $\dot{T}$ no Gauss mais próximo de x_f (no código: `argmax(xg)`). Para acoplar $\dot{T}_f$ à massa de fato, use Gauss–Lobatto ou DIBEM.

Método das linhas (implementação)

Estado $y = (T_f, T_1^g, \dots, T_{n_i}^g) \in \mathbb{R}^{N-1}$.

A matriz A do sistema em $z = (Q_0, \dot{T}^g)$ depende só da malha (Gb, Mg) , **não** de y . Fatora-se **uma vez**; a cada RHS só monta o residual $r(y)$ e faz $z = F \setminus r$.

```

"""Fator LU de A (constante no tempo). Linhas de colocação: x0 + Gauss."""
function bem_heat_factor(mesh)
    (; Gb, Mg, ni, N) = mesh
    rows = vcat(1, 3:N)          # N-1 equações
    nz = 1 + ni
    A = zeros(nz, nz)
    for (eq, p) in enumerate(rows)
        A[eq, 1] = -Gb[p, 1]      # coluna Q0
        A[eq, 2:end] .= Mg[p, :] # colunas Tg
    end
    return lu(A), rows
end

"""RHS: só r(y); resolve com fator pré-computado F."""
function bem_heat_rhs!(dy, y, p, t)
    mesh, TL, F, rows = p
    (; Hb, nc, ni, xg) = mesh
    Tf = y[1]

```

```
Tg = y[2:end]
@assert length(Tg) == ni

r = zeros(1 + ni)
for (eq, pidx) in enumerate(rows)
    r[eq] = -(Hb[pidx, 1] * TL + Hb[pidx, 2] * Tf)
    if pidx > nc
        r[eq] -= Tg[pidx - nc]
    end
end
z = F \ r
Tdotg = z[2:end]

dy[1] = Tdotg[argmax(xg)]           #  $\dot{T}_f \approx \dot{T}$  no Gauss mais à direita
dy[2:end] .= Tdotg
return nothing
end

function solve_bem_heat(; L=1.0, κ=1.0, TL=0.0, tspan=(0.0, 0.5), nq=8,
                        T0 = nothing)
    mesh = bem1d_M(0.0, L; κ=κ, nq=nq)
    F, rows = bem_heat_factor(mesh)
```

```
if T0 === nothing
    T0 = x -> TL + (0.0 - TL) * (x / L)
end
y0 = vcat(T0(L), T0.(mesh.xg))
prob = ODEProblem(bem_heat_rhs!, y0, tspan, (mesh, TL, F, rows))
sol = solve(prob, Tsit5(); reltol=1e-8, abstol=1e-8)
return sol, mesh
end

function profile_at(solB, mesh, TL, t)
    y = solB(t)
    Tf, Tg = y[1], y[2:end]
    x = vcat(mesh.x0, mesh.xg, mesh.xf)
    T = vcat(TL, Tg, Tf)
    perm = sortperm(x)
    return x[perm], T[perm]
end
```

Exemplo com solução analítica

Problema (compatível com as BC mistas do código: Dirichlet à esquerda, Neumann nulo à direita):

$$\begin{aligned} T_t &= \kappa T_{xx}, \quad 0 < x < L, \quad t > 0, \\ T(0, t) &= 0, \quad \partial_x T(L, t) = 0, \\ T(x, 0) &= \sin(\lambda_0 x), \quad \lambda_0 = \pi/(2L). \end{aligned}$$

Autofunção de ∂_{xx} com essas BC: $\sin(\lambda_n x)$, $\lambda_n = (2n + 1)\pi/(2L)$. O modo fundamental ($n = 0$) evolui exatamente como

$$T_{\text{exata}}(x, t) = \sin(\lambda_0 x), e^{-\kappa \lambda_0^2 t}, \quad \lambda_0 = \pi/(2L).$$

Com $L = 1$, $\kappa = 1$: $\lambda_0 = \pi/2$, $T(x, t) = \sin(\pi x/2), e^{-(\pi/2)^2 t}$.

```
L, κ = 1.0, 1.0
λ0 = π / (2L)
T_exact(x, t) = sin(λ0 * x) * exp(-κ * λ0^2 * t)
TL = 0.0
tspan = (0.0, 0.5)

solB, mesh = solve_bem_heat(; L=L, κ=κ, TL=TL, tspan=tspan, nq=8,
    T0 = x -> sin(λ0 * x))

# --- T(L,t): BEM × analítico ---
ts = range(tspan...; length=80)
TR_bem = [solB(t)[1] for t in ts]
```

Exemplo com solução analítica

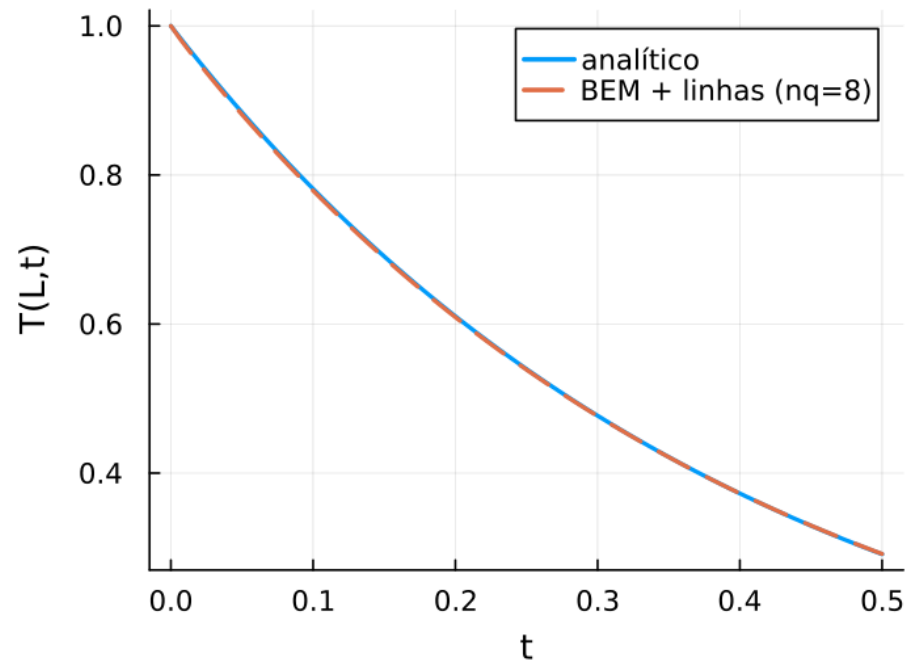
```
TR_ex = [T_exact(L, t) for t in ts]

p1 = plot(ts, TR_ex; lw=2, label="analítico", xlabel="t", ylabel="T(L,t)",
          title="Ponta direita")
plot!(p1, ts, TR_bem; lw=2, ls=:dash, label="BEM + linhas (nq=$(mesh.ni))")

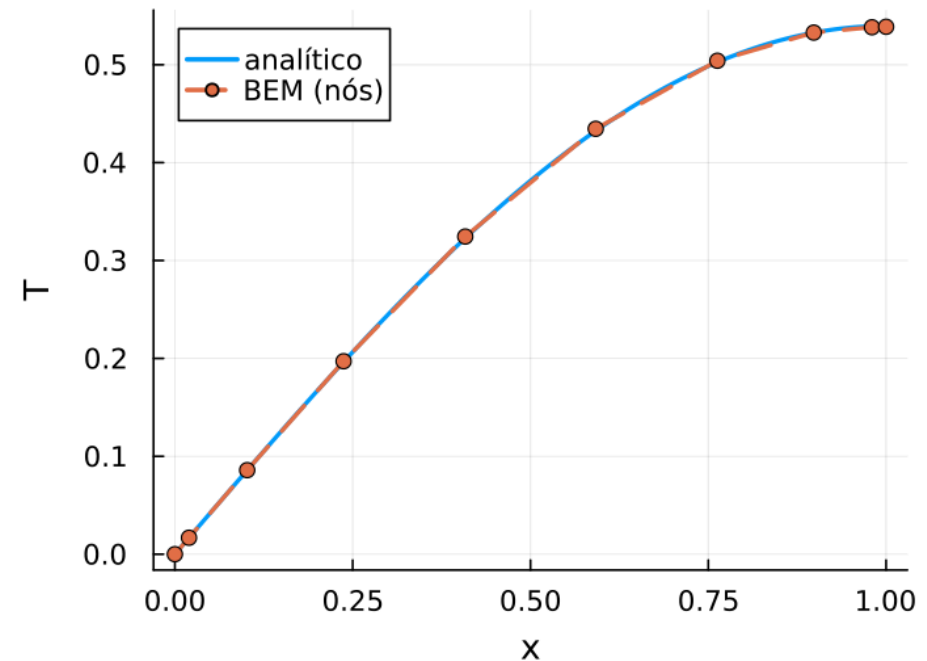
# --- perfil em t fixo ---
t_snap = 0.25
xp, Tp = profile_at(solB, mesh, TL, t_snap)
xx = range(0, L; length=200)
p2 = plot(xx, T_exact.(xx, t_snap); lw=2, label="analítico",
          xlabel="x", ylabel="T", title="Perfil t = $t_snap")
plot!(p2, xp, Tp; marker=:circle, lw=2, ls=:dash, label="BEM (nós)")

plot(p1, p2; layout=(1, 2), size=(900, 350))
```

Ponta direita



Perfil t = 0.25



```
# --- erros ---
err_TR = maximum(abs, TR_bem .- TR_ex)
err_prof = maximum(abs, Tp .- T_exact.(xp, t_snap))
@show err_TR err_prof

# --- estudo com nq ---
println("nq\terr_T(L,t)\terr_perfil(t=$t_snap)")
```

BEM 1D × analítico

```

for nq in (2, 4, 6, 8, 12)
    soln, mn = solve_bem_heat(; L=L, κ=κ, TL=TL, tspan=tspan, nq=nq,
        T0 = x -> sin(λ0 * x))
    eL = maximum(abs, [soln(t)[1] - T_exact(L, t) for t in ts])
    xq, Tq = profile_at(soln, mn, TL, t_snap)
    ep = maximum(abs, Tq .- T_exact.(xq, t_snap))
    println("$nq\t$eL\t$ep")
end

```

Leitura esperada: com o modo fundamental nas BC certas, o erro em $T(L, t)$ e no perfil cai ao aumentar n_q (mais fontes interiores = melhor M^g), até saturar pelo modelo 1D/SF estacionária e pela amarração de $\dot{T}_f$. Compare sempre com a curva $e^{-(\pi/2)^2 t}$ na ponta ($T(L, 0) = 1$).

Por que essa CI? $\sin(\pi x/(2L))$ some em $x = 0$ e tem derivada nula em $x = L$, logo respeita as BC para todo t na solução exata. Isso isola o erro da **semi-discretização BEM**, não de BC incompatíveis.

Checklist.

1. $N = n_c + n_i$ colocações: 2 extremos + todos os Gauss.
2. $M^g: N \times n_i$, $M_{pj}^g = w_j \kappa^{-1} T^*(x_j^g, x_d^p)$ — coluna = fonte de domínio no Gauss j .
3. $H^b, G^b: N \times 2$.
4. Fator LU de A ($N - 1$) **uma vez**; cada RHS só monta $r(y)$ e faz $F \setminus r$ para $(Q_0, \dot{T}^g)$.
5. 2D: o mesmo papel dos Gauss é o dos pontos interiores DIBEM.

Checklist MDF × BEM (linhas)

1. Espaço $\rightarrow$ PVI/DAE no tempo.
2. BEM: estado = contorno livre + T em cada Gauss (fonte interior).
3. Matriz espacial densa de ordem $\sim n_c + n_i$, **fatorada uma vez**; residual a cada passo.
4. Integrador no tempo intercambiável (Tsit5, RK4, Houbolt).
5. $n_i \uparrow$ enriquece domínio e tamanho do sistema.

Entregue para cada item: código Julia (Plots), figuras pedidas e **duas ou três frases** de interpretação. Use os solucionadores e rotinas do capítulo (euler, rk4, diffmat, bem1d_M, solve_bem_heat, ...).

1. **E1 – Ordem de Euler e RK4.** Considere o PVI

$$u' = -2tu, \quad u(0) = 2, \quad t \in [0, 2],$$

com solução exata $\hat{u}(t) = 2e^{-t^2}$.

(a) Com euler e rk4, tome $n = 10 \cdot 2^d$ para $d = 1, \dots, 8$ e calcule o erro no tempo final

$$e(n) = | u_n - \hat{u}(t_n) |.$$

(b) Em um único gráfico log-log ($n \times e(n)$), plote as duas curvas e sobreponha retas de referência $C_1 n^{-1}$ e $C_4 n^{-4}$ (ajuste C_1, C_4 no primeiro n de cada método).

(c) Com base nas inclinações, confirme as ordens e diga a partir de qual n o RK4 deixa de ganhar (saturação por aritmética ou pela referência).

2. **E2 – De 2ª ordem a sistema.** O problema

$$u'' + 9u = 9t, \quad u(0) = 1, \quad u'(0) = 1, \quad t \in [0, 2\pi]$$

tem solução $\hat{u}(t) = t + \cos(3t)$.

(a) Escreva o sistema de 1ª ordem em $\mathbf{y} = (u, u')$ e a função $f(\mathbf{y}, t)$ correspondente.

(b) Integre com rk4 ($n = 200$) e com Tsit5 (DifferentialEquations). Reporte

$$e = | u(2\pi) - \hat{u}(2\pi) |$$

nos dois casos.

(c) Plote $u(t)$ numérico (RK4) e $\hat{u}(t)$ no mesmo eixo. Onde o erro se concentra ao longo de t ?

3. E3 – PVC 1D com diffmat. Resolva

$$-T'' = 1, \quad x \in (0, 1), \quad T(0) = T(1) = 0,$$

cuja solução exata é $T(x) = x(1 - x)/2$.

(a) Use diffmat e imponha Dirichlet por substituição de linhas. Para $n \in \{10, 20, 40, 80\}$ (intervalos), calcule

$$e_\infty(h) = \max_i | T(x_i) - T_h(x_i) |, \quad h = 1/n.$$

(b) Plote e_∞ vs h em escala log-log e estime a ordem p em $e = O(h^p)$.

(c) Para $n = 40$, plote T exata e T_h juntas. O erro máximo ocorre no centro ou perto das bordas? Por quê, dada a ordem das fórmulas em D_{xx} ?

4. E4 – MDF × BEM × solução analítica (calor 1D). Considere o mesmo problema do exemplo do texto:

$$\begin{aligned} T_t &= T_{xx}, \quad 0 < x < 1, \\ T(0, t) &= 0, \quad \partial_x T(1, t) = 0, \\ T(x, 0) &= \sin(\pi x/2), \end{aligned}$$

com solução exata

$$T(x, t) = \sin(\pi x/2), e^{-(\pi/2)^2 t}.$$

MDF (método das linhas). Use `diffmat` em $[0, 1]$ com n intervalos, imponha $T(0, t) = 0$ e a Neumann $\partial_x T(1, t) = 0$ (substitua a última linha de D_{xx} por a linha correspondente de D_x , ou uma condição equivalente de ordem 2). Integre no tempo com `tsit5` até $t = 0.5$.

BEM (método das linhas no contorno). Use `solve_bem_heat` / `bem1d_M` com n_q Gauss como fontes interiores (fator LU pré-computado).

(a) **Erros em relação ao analítico.** Para MDF com $n \in \{40, 80, 160\}$ e BEM com $n_q \in \{2, 4, 6, 8, 12\}$, reporte

$$e_L = \max_{t \in [0, 0.5]} |T(1, t) - T_{\text{exata}}(1, t)|$$

e o erro máximo do perfil em $t = 0.25$. Organize em duas tabelas (MDF e BEM).

(b) **Figuras.** Em $t = 0.25$, plote no **mesmo** gráfico: solução exata, perfil MDF (melhor n) e perfil BEM (melhor n_q). Em outro gráfico: $T(1, t)$ exata, MDF e BEM ao longo do tempo.

(c) **Comparação MDF × BEM.** Com os números e as figuras:

- qual método atinge erro $< 10^{-3}$ em e_L com menos graus de liberdade no **estado** do PVI? (MDF: $\sim n - 1$ interiores; BEM: $1 + n_q$)
- o erro do BEM satura ao subir n_q ? O do MDF satura ao subir n ?
- comente o custo por passo: fator esparsa/aplicação de D_{xx} vs. backsolve denso $(N - 1) \times (N - 1)$ já fatorado.

(d) **Pergunta teórica:** se $M^g \equiv 0$ no BEM, o que resta das equações de colocação? Por que some a dinâmica com a SF estacionária de Laplace?

5. E5 — Propagação de onda (MDF + comparação; esboço BEM).

Teoria mínima (releia a § “Equação da onda”). A EDP $u_{tt} = c^2 u_{xx}$ é de **segunda ordem no tempo**. Para usar euler / rk4 / Tsit5 (feitos para $y' = f(t, y)$), reduza a ordem:

$$v = u_t, \quad u_t = v, \quad v_t = c^2 u_{xx}.$$

No espaço, troque u_{xx} por $D_{xx} \mathbf{u}$ (diffmat). O estado do PVI é

$$\mathbf{y} = (\mathbf{u}, \mathbf{v}).$$

Em extremos com Dirichlet **fixo** $u = 0$, imponha também $v = u_t = 0$ nesses nós.

Problema.

$$\begin{aligned} u_{tt} &= c^2 u_{xx}, \quad 0 < x < 1, \quad 0 < t \leq 2, \\ u(0, t) &= u(1, t) = 0, \\ u(x, 0) &= \sin(\pi x), \quad u_{t(x, 0)} = 0, \end{aligned}$$

com $c = 1$ e solução exata

$$u_{\text{exata}}(x, t) = \sin(\pi x) \cos(\pi t).$$

(a) **Implementação MDF.** Usando o esqueleto `wave_rhs!` / `solve_wave_mdf` (ou equivalente seu) com `diffmat` e `Tsit5`, resolva para $n \in \{50, 100, 200\}$ intervalos. Em cada n , calcule

$$e_{\infty}(t) = \max_i |u_h(x_i, t) - u_{\text{exata}}(x_i, t)|$$

Exercícios

nos instantes $t \in \{0.5, 1.0, 1.5, 2.0\}$. Monte uma tabela (n, t, e_∞) .

(b) **Figuras.**

- Perfis $u(x, t)$ em $t = 0, 0.5, 1.0, 1.5$ (MDF com o melhor n) sobrepostos à exata (linhas tracejadas).
- Série temporal no ponto médio: $u(1/2, t)$ numérico vs $\cos(\pi t)$ (exata, pois $\sin(\pi/2) = 1$).

(c) **Dispersão e refinamento.** O erro cresce com t mesmo com BC e CI “perfeitas” no modo fundamental? Se sim, atribua a **dispersão numérica** de D_{xx} (a velocidade numérica do modo depende de h). Mostre que e_∞ em $t = 2$ cai ao aumentar n e estime a ordem aparente em h .

(d) **Comparação qualitativa com o calor (E4).** Em uma frase cada: (i) o que acontece com a “energia” / amplitude do modo no calor vs na onda; (ii) restrição de passo no tempo explícito ($\Delta t = O(h^2)$ vs $O(h)$).

(e) **BEM — só estrutura (sem obrigar código).** Escreva a forma matricial esperada no contorno

$$M\mathbf{u}'' + H\mathbf{u} = G\mathbf{q}$$

e diga: o que são as incógnitas em cada extremo se $u(0, t) = u(1, t) = 0$? O método das linhas no contorno atuaria sobre quais componentes de z ? Em que o bloco M aqui difere do M^g do calor (segunda derivada temporal vs primeira)?

(f) **Opcional (Houbolt).** Com o extra do capítulo, integre o oscilador modal equivalente $U'' + (c\pi)^2 U = 0$ (amplitude do modo $\sin(\pi x)$) com Houbolt e compare $U(t)$ a $\cos(c\pi t)$. Relacione com o item (b) no ponto médio.

Extra: multi-passo AB4 e Houbolt

RK avalia f várias vezes por passo **sem** histórico. Métodos de **múltiplos passos** reutilizam $f_i = f(t_i, u_i)$ passados. Ficam no fim porque a trilha BEM já está completa com Euler/RK/Tsit5; use-os quando quiser menos avaliações de f ou integradores da dinâmica estrutural.

$$u_{i+1} = u_i + h \left(\frac{55}{24} f_i - \frac{59}{24} f_{i-1} + \frac{37}{24} f_{i-2} - \frac{9}{24} f_{i-3} \right).$$

Precisa de 3 valores iniciais (starter: RK4 num trecho curto).

```
function ab4(ivp, n)
    a, b = ivp.tspan
    h = (b - a) / n
    t = [a + i*h for i in 0:n]
    u0 = ivp.u0 isa Number ? float(ivp.u0) : float.(ivp.u0)
    U = Vector{typeof(u0)}(undef, n + 1)
    _, Us = rk4(ODEProblem(ivp.f, ivp.u0, (a, a + 3h), ivp.p), 3)
    U[1:4] .= Us
    σ = [-9, 37, -59, 55] ./ 24
    F = [ivp.f(U[i], ivp.p, t[i]) for i in 1:4]
    for i in 4:n
        U[i+1] = U[i] + h * sum(σ[j] * F[j] for j in 1:4)
        F = (F[2], F[3], F[4], ivp.f(U[i+1], ivp.p, t[i+1]))
    end
    return t, U
end
```

Houbolt (2ª ordem no tempo)

Comum em dinâmica estrutural e em BEM elástico transiente. Aproxima

$$u''_{n+1} = \frac{2u_{n+1} - 5u_n + 4u_{n-1} - u_{n-2}}{h^2},$$

$$u'_{n+1} = \frac{11u_{n+1} - 18u_n + 9u_{n-1} - 2u_{n-2}}{6h}.$$

Implícito: u_{n+1} entra nos dois lados quando a EDO é $u'' = f(t, u, u')$. Amortece altas frequências (rígido).

Esqueleto para o oscilador $u'' + \omega^2 u = 0$ (estado escalar; starter com RK4 na forma 1ª ordem):

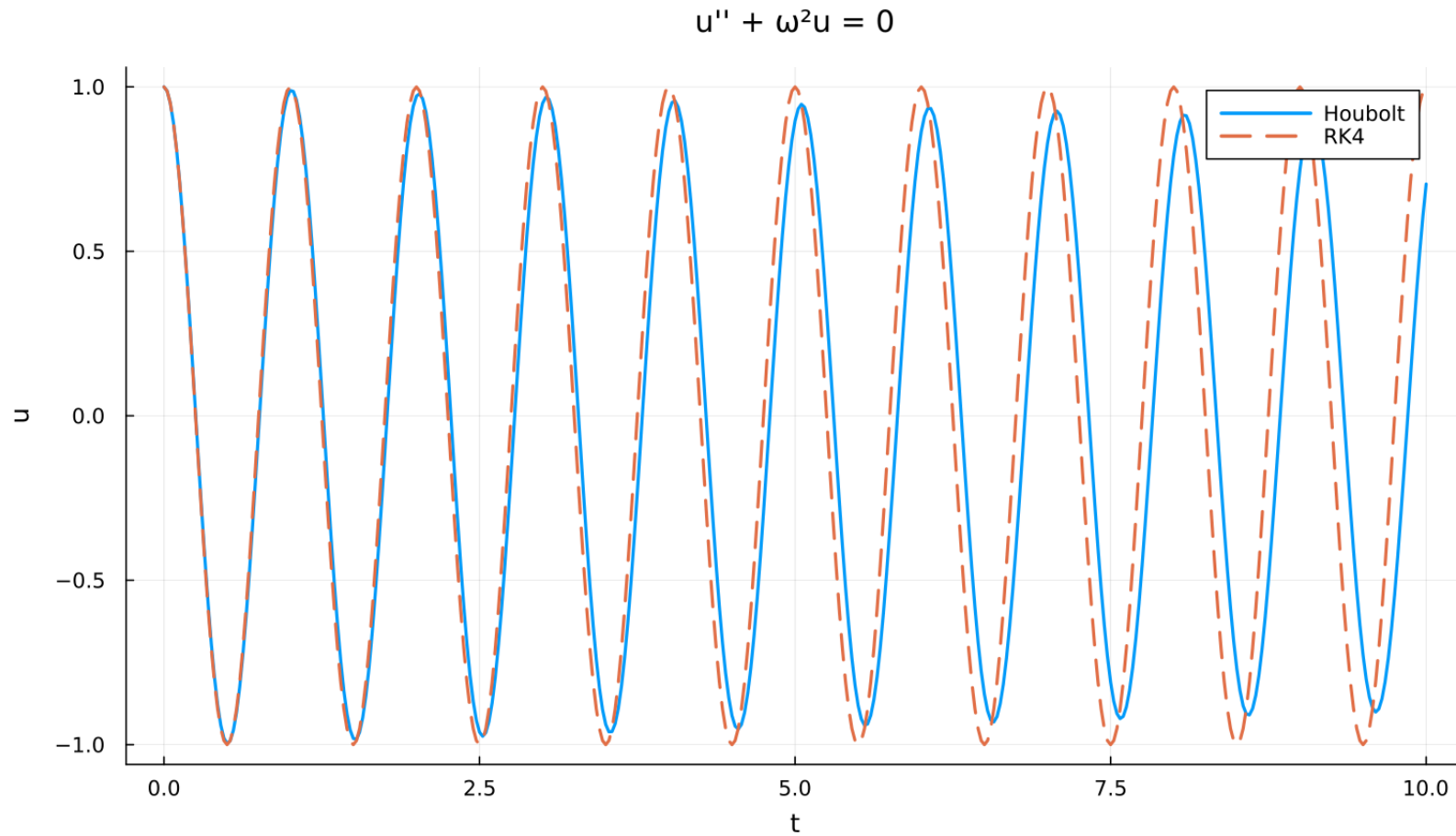
```

"""Houbolt para u'' + ω² u = 0, n passos em [0, tf]."""
function houbolt_oscillator(ω, u0, v0, tf, n)
    h = tf / n
    t = collect(range(0, tf; length=n+1))
    u = zeros(n + 1)
    # starter: RK4 no sistema (u,v)
    fsys = (y, p, τ) -> [y[2], -ω^2 * y[1]]
    _, Ys = rk4(ODEProblem(fsys, [u0, v0], (0.0, 2h), nothing), 2)
    u[1] = u0
    u[2] = Ys[2][1]
    u[3] = Ys[3][1]
    for i in 3:n
        # (2/h²) u_{i+1} + ω² u_{i+1} = (5 u_i - 4 u_{i-1} + u_{i-2}) / h²
    end
end

```

```
    rhs = (5u[i] - 4u[i-1] + u[i-2]) / h^2
    u[i+1] = rhs / (2/h^2 + ω^2)
end
return t, u
end

ω = 2π
tH, uH = houbolt_oscillator(ω, 1.0, 0.0, 10.0, 400)
tR, UR = rk4(ODEProblem((y,p,τ)->[y[2], -ω^2*y[1]], [1.0, 0.0], (0.0, 10.0), nothing), 400)
plot(tH, uH; label="Houbolt", lw=2)
plot!(tR, [y[1] for y in UR]; label="RK4", ls=:dash, lw=2,
      xlabel="t", ylabel="u", title="u'' + ω^2u = 0")
```



Adams–Moulton / predictor–corretor: leitura exterior (não são necessários para a trilha BEM deste capítulo).